
PHDATA INTELLIGENCE PLATFORM • SENIOR APPLIED AI ENGINEER CHALLENGE

ServeWell IT Support Agent Suite

An agentic triage → grounded resolution → human-approved action system,
proven on a narrow slice of ServeWell's L1 support load.

ROUTING ACCURACY

90.2%

RUNBOOK HIT-RATE

100%

GROUNDED REPLIES

100%

RESPONSE P95

3.9s

Measured on 220 labelled tickets against the challenge answer key • reproducible with one command

Two audiences, one system

The brief asks for a walkthrough a non-technical stakeholder can follow and a technical reviewer can trust. These slides are marked so you can skip to your half — but the middle section is the one I most want probed.

FOR THE BUSINESS UNIT VP

- 3 — The problem, in ServeWell's own numbers
- 4 — What we built, in plain language
- 17 — Business impact and what it rests on
- 18 — What I would not yet claim

FOR THE DIRECTOR OF TECHNOLOGY

- 5-6 — Component architecture and request path
- 7-11 — Agents, RAG, tools, actions, guardrails
- 12-15 — Evaluation, a measurement failure, tuning
- 16 — Technical trade-offs and where I drew the line

Every number in this deck is measured by a harness in the repo, or explicitly labelled as an assumption. Nothing is estimated and presented as measured.

THE PROBLEM

What ServeWell told us is breaking

30–45 min

to first meaningful response

during peak hours, when a store is losing revenue

60–70%

of tickets are already known

they map to an existing runbook, but still need a human to read it

2,500+

locations, inconsistent service

some stores get quick help; others bounce between L1 and L2

AND THE COST LANDS IN THREE PLACES

- **Revenue:** Store downtime reduces revenue and customer satisfaction directly.
- **People:** L1 cost and burnout are rising, with high attrition in the support centre.
- **Capacity:** L2 engineers are absorbing basic tickets that should never have reached them.

Note the shape of this: the knowledge to fix most of these tickets already exists. The bottleneck is getting it to the right person fast, and knowing when not to try.

What the system actually does

- 1 A ticket arrives**
“The PIN pad on terminal 1 is not responding.”
- 2 It decides who should handle it**
Straight to a human L2 engineer, or guided at L1
- 3 It finds the right runbook**
and reads the store, asset and warranty records
- 4 It writes the steps back**
grounded in that runbook — never invented
- 5 It can offer to act**
but a human approves before anything runs

The one design rule

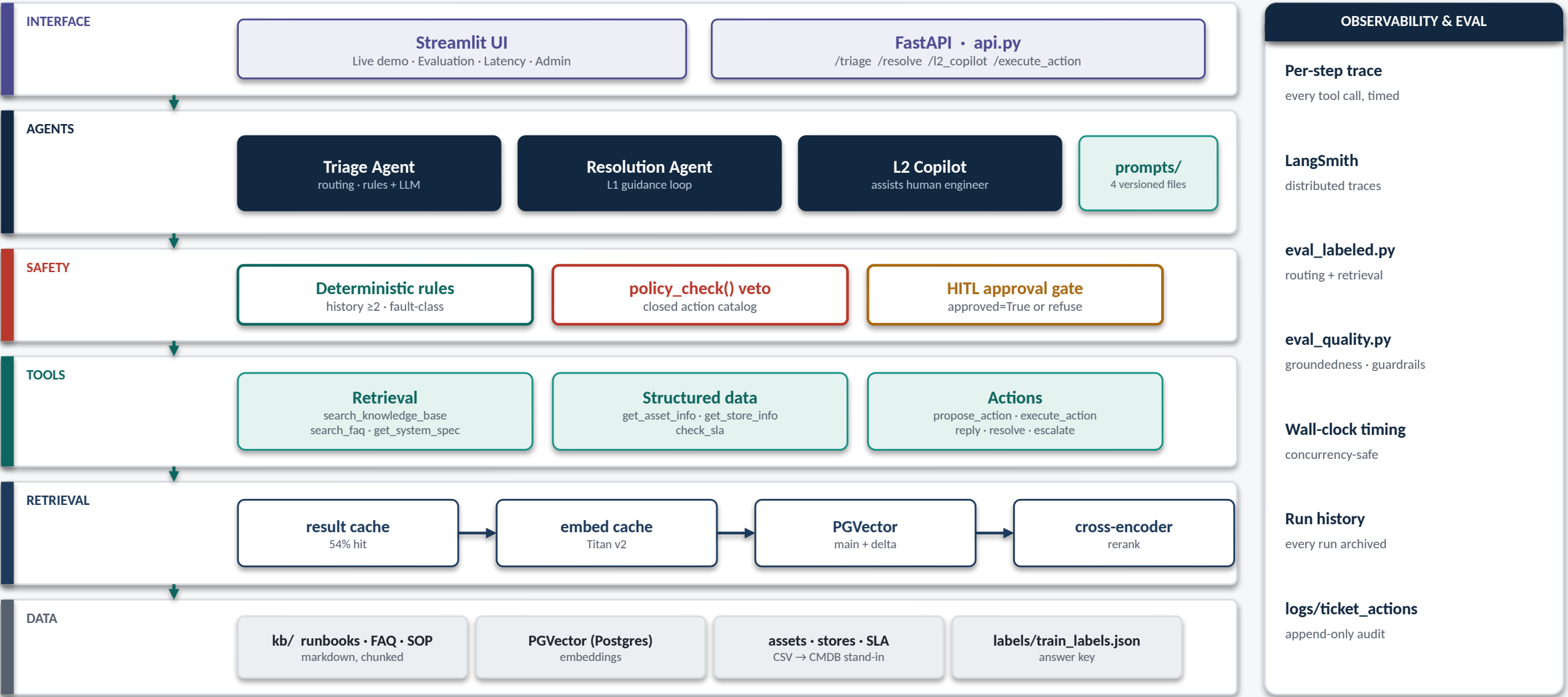
The agent is allowed to be wrong about advice, because a human reads it before acting on it.

It is never allowed to be wrong about an action, because nothing runs without an explicit human approval.

That asymmetry is why the autonomy line sits where it does.

Advice is reversible. Actions are not.

Component-level view



What happens to one ticket, and where the time goes



TWO THINGS THAT MATTER HERE

- **They overlap.** Retrieval, FAQ, the spec-sheet lookup and the CMDB enrichment all run concurrently in one thread pool. Summing the per-step timings over-counts real elapsed time by 25–65%, so the harness stamps true wall-clock instead.
- **Escalation short-circuits.** A ticket routed to L2 never enters the resolution loop at all — triage is a single LLM call on ticket text, with no retrieval.

MEDIAN

0.62s
half of tickets

P95

3.9s
worst realistic case

MEAN

1.17s
excl. one transient stall

VS. TODAY

30–45 min
ServeWell's stated baseline

The agents, and how work is orchestrated

Agent	Responsibility	Control flow	Tool access
Triage Agent	Decide the queue: L1_GUIDED, L2_ESCALATION or NON_IT	Deterministic rules first, one LLM call only for genuine judgement calls	No tools. Ticket text only.
Resolution Agent	Guide the store through the runbook, or hand off	Bounded loop: ≤5 turns, ≤2 KB searches, 1 action proposal	Retrieval + CMDB + actions
L2 Copilot	Assist the human engineer on escalated tickets	Open-ended chat, ≤2 searches per question, no further escalation	Same tools, no HITL gate

WHY A CUSTOM LOOP RATHER THAN A FRAMEWORK

- **Defensibility:** Every control decision — the search budget, the turn cap, the approval gate — is a visible line of Python I can point at in this room. With LangGraph or CrewAI those same limits live inside framework config, and 'why did it stop there?' becomes a harder question to answer live.
- **Proportionality:** The orchestration is ~200 lines. A framework would have been justified if we needed persistence, retries across sessions, or multi-agent negotiation. We need none of those for one incident type.
- **Measured need:** Bounded loops, not open-ended autonomy. Instruction-following alone did not hold — the model reworded queries and searched 4–6 times per ticket until the cap was enforced in code.

Triage: when a simple rule beats the AI

One question decides most tickets: how many times has L1 already worked this one and failed?

Nobody has tried yet

93 tickets

AI decides

gets it right 67% of the time

L1 tried once

40 tickets

AI decides

gets it right 70% of the time

L1 tried twice or more

87 tickets

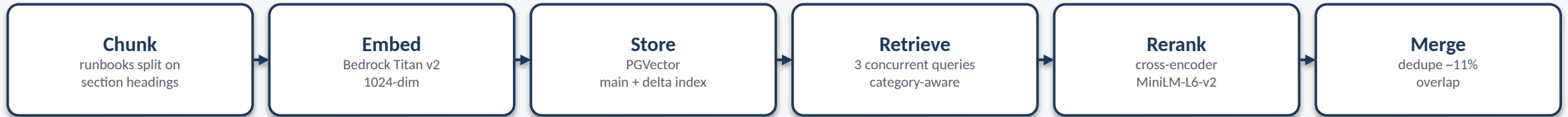
Rule decides

the rule is right 97% — the AI only 92%

In plain terms: if L1 has already worked a ticket twice and it is still open, you do not need AI to tell you it needs an expert. A one-line rule is more accurate than the model there — and it is free, instant, and always gives the same answer.

Being straight with you: these thresholds were set using the same 220 tickets they are scored on, so treat 90% as a ceiling, not a promise.

How retrieval is structured, and how answers stay grounded



- **Three queries, not one.** A blended category + subject + description query, plus a subcategory-only query, plus an FAQ-filtered query — run concurrently. The blend alone missed network runbooks on POS/Network tickets because 'POS terminal' dominates the embedding.
- **One deterministic lookup.** Spec sheets read like reference tables, not symptoms, so they rank poorly against symptom queries. `get_system_spec` attaches the right one by `system_version` instead of hoping search finds it.
- **Provenance survives.** Every chunk keeps its “--- Document Source: X ---” header through merging, so every claim in a reply can be traced back to a file.

RUNBOOK HIT-RATE

100%

correct doc retrieved, n=75

RECALL@K

72.8%

of all labelled docs

GROUNDEDNESS

100%

specifics traced to source

RERANKER LIFT

+2.7pp

hit-rate vs. no rerank

Recall@k is 73%, not 100%: the key lists ~4–5 docs per ticket and we retrieve the decisive one plus some. Hit-rate is the metric that matches the job.

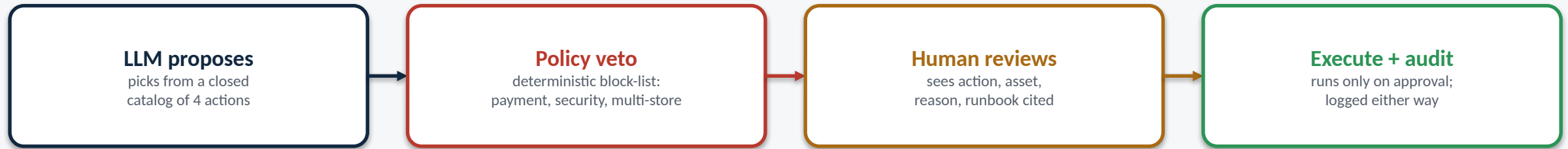
Structured data: when it calls a tool, when it reasons

What	Tool call or reasoning?	Why it is drawn there
Asset, store, SLA, warranty	Always fetched, before the model runs	Deterministic — never left to the model to remember to ask. ~18ms.
The relevant runbook	Always pre-fetched, 3 concurrent queries	The model reads results; it does not decide whether to look.
System spec sheet	Deterministic lookup on system_version	Semantic search ranks these poorly; a keyword map is simply better.
A second KB search	Model's choice — capped at one	Genuine judgement: 'is the pre-fetched runbook actually about this?'
Which steps to give	Model reasons over retrieved text	This is the part that needs a language model at all.
Whether to act	Model proposes; policy and a human dispose	Never the model's call alone.

The principle

If the answer is a lookup, do the lookup — do not spend a model turn deciding to. Reserve the model for the parts that are genuinely ambiguous. Structured facts are injected as authoritative and the prompt forbids contradicting them, so the agent cannot advise a warranty replacement on an out-of-warranty asset, and says so plainly when an asset is missing from the CMDB rather than inventing it.

Actions: the agent proposes, a human disposes



REFUSAL IS THE DEFAULT, NOT AN EDGE CASE

- **The gate is code:** `execute_action()` refuses unless `approved=True`, and that check lives in the tool, not the UI — so no future caller (a script, a retry path, another agent) can reach an actuator by skipping the screen that was meant to ask.
- **Closed vocabulary:** Four reversible, single-asset operations. The model cannot emit a free-text command, and cannot propose a printer restart for a router — the catalog is typed to asset class.
- **Policy runs first:** Categories the SOP marks immediate-escalate (payment, outage, security, database, multi-store) are refused before a human is ever asked. A confidently wrong proposal never becomes a button.
- **Everything is logged:** Proposed, refused and executed attempts all append to `logs/ticket_actions.jsonl` with actor and timestamp — the ITSM audit trail, in POC form.

How I know it is correct, safe and reliable

ROUTING ACCURACY

90.2%

mean of 4 runs · vs answer key

RUNBOOK HIT-RATE

100%

correct doc retrieved

RECALL@K

72.8%

of all labelled docs

GROUNDNESS

100%

148 specifics, 0 unsupported

GUARDRAIL PASS

100%

no budget or policy breach

RESPONSE P95

3.9s

true wall-clock

WHAT MAKES THESE TRUSTWORTHY

- **Nothing is simulated:** Every run calls the live agent over HTTP for all 256 tickets — no mocks, no cached answers. One command reproduces it.
- **Self-describing guardrails:** The agent publishes its own search budget into the trace, so the guardrail checker grades against the real policy instead of a hardcoded copy that drifted out of sync twice.
- **Latency measured honestly:** Latency comes from the same calls made for accuracy, stamped as true wall-clock — concurrent runs are flagged invalid and excluded from the trend history rather than quietly inflating p95.

We were grading against the wrong answer key

THE SETUP

Every ticket ships with `escalation_flag` — what an upstream system marked. Separately, `labels/train_labels.json` carries `correct_routing`: the actual answer key.

THE BUG

Our evaluator scored routing against `escalation_flag`. It agrees with the real key only 50.5% of the time — worse than a coin flip on the one signal that mattered.

HOW IT SURFACED

A routing number that looked plausible (72%) did not survive the question “is this graded against the right file?”. It was not.

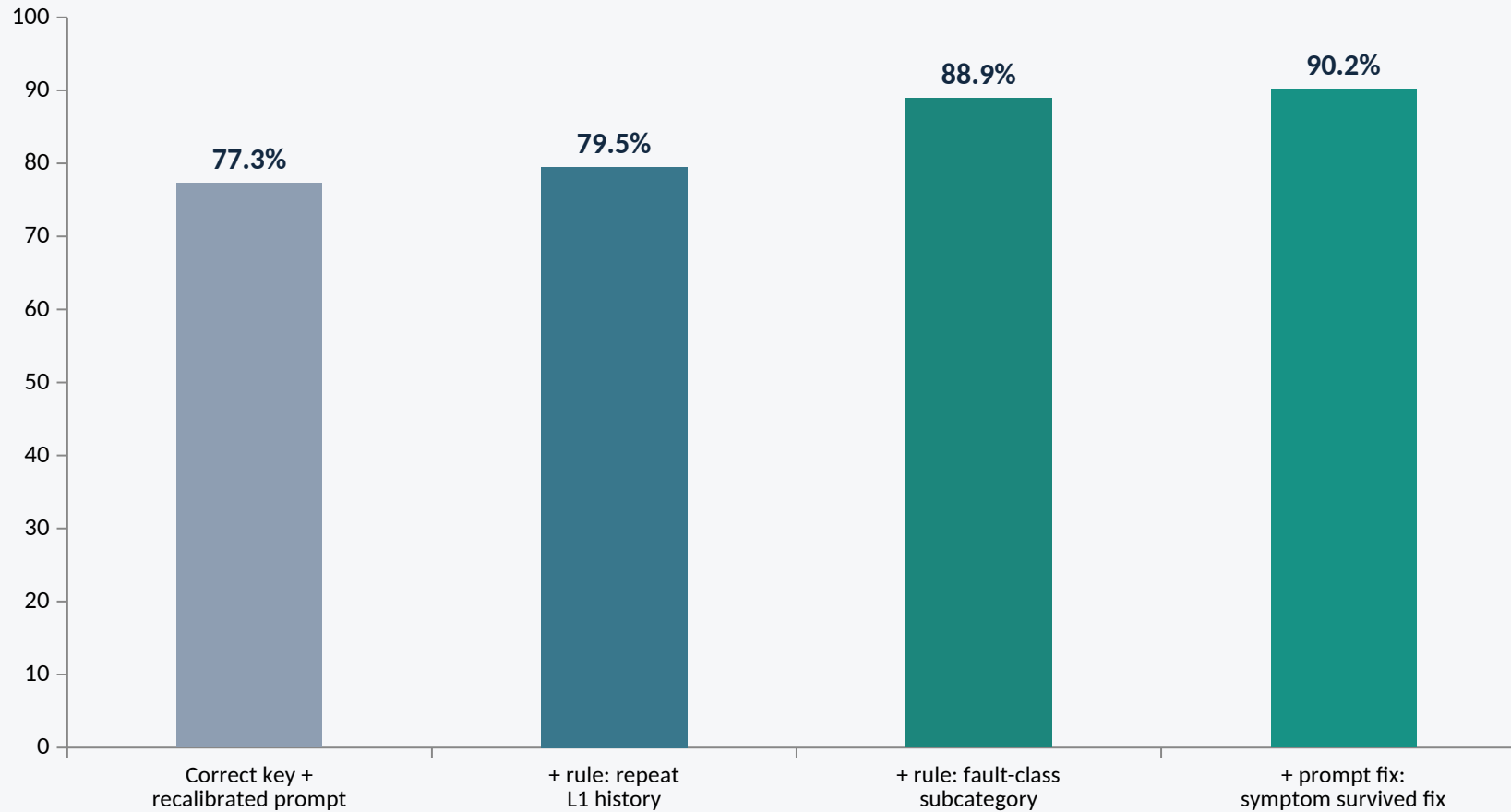
THE FIX

Re-pointed every evaluator, the UI and the ROI model at `correct_routing`, then recalibrated triage against the real target distribution — 65% of tickets should escalate, not ~50%.

WHY IT IS ON A SLIDE

A POC that cannot detect its own measurement error is not evidence of anything. This is the check that makes the other numbers worth reading.

Routing accuracy — four measurements, from the corrected baseline



EACH STEP WAS MEASURED

Not one prompt rewrite. Each gain came from reading the failures, forming a hypothesis, and testing it at n=220.

TOTAL GAIN

+11 pts

79.5% → 90.2%

ESCALATION CALIBRATION

66% vs 65%

predicted vs expected

One attempt overshoot to 83% and was reverted.

Making it fast — and measuring it honestly

What was slow	Why	Fix	Effect
Summed step timings	Over-counted 25–65%: concurrent calls double-counted	True wall-clock stamped on every exit path	correctness
Duplicate action proposals	Model re-proposed the same action, burning LLM turns	Capped at one successful proposal per ticket	–27% avg
Retry on invalid actions	Refusal invited another guess for assets with no actions	Refusal now names that none exist	outlier removed
Repeated identical queries	Re-embedded, re-searched, re-ranked every time	Full result cache — 54% hit rate warm	–25% p95

AVG RESPONSE

1.9s → 1.2s

–39%

P95 RESPONSE

5.8s → 3.9s

–34%

WORST CASE

64s → 5.9s

stall removed

ACCURACY

unchanged

no trade made

The caching result improves with volume: hit rate went 0% → 18% → 54% as the cache warmed across runs. At 400 tickets/day over a stable ~45-subcategory taxonomy, the same queries recur constantly — this gets better in production, not worse.

The trade-offs I made, and what I gave up

Decision	What I chose	Why, and what it cost
Model	Bedrock nvidia-nemotron-nano-3-30b	The only model this AWS account can reach — verified by AccessDenied on every Claude/Nova/Llama candidate. A small model made the guardrails earn their place.
Orchestration	Custom ~200-line loop, not LangGraph/CrewAI	Every limit is a line of Python I can defend live. Gave up: persistence, retries, a visual graph.
Retrieval	PGVector + Titan v2 + cross-encoder rerank	Rerank costs ~0.3s and is CPU-bound, but buys +2.7pp hit-rate. Diverged from the plan's ChromaDB + MiniLM.
Autonomy line	Propose-only; execution needs approved=True	Slower than auto-remediation. Chosen because a wrong action in 2,500 stores is unrecoverable; wrong advice is not.
Triage design	Deterministic rules where measured better	Less elegant than 'the LLM decides'. But at 2+ notes the rule beats the model 96.6% to 92.0%.
Scope	One incident family, deep, over six shallow	The brief rewards a POC defended deeply. Ticket write-back is the visible gap.

Prompts now live as versioned files in prompts/ — extracted byte-identically, so a prompt change reads like a behaviour change in a diff, which is what it is.

What this is worth, and what each number rests on

MEASURED STATED BY SERVEWELL YOUR ASSUMPTION			
L1 COST AVOIDED / YR		L1 HOURS RETURNED / YR	TICKETS DEFLECTED / DAY
\$202k		9,198	84
at \$22/hr fully loaded		≈ 4.4 FTE at 2,080 hrs/yr	of 140 L1-eligible (35% share)
TIME TO FIRST RESPONSE		38 min → 1.2s	
measured end to end			
Tickets resolvable at L1	35%	MEASURED	answer key, 220 tickets
Runbook retrieved correctly	100% hit-rate	MEASURED	vs. answer key
Guidance grounded in a runbook	100%	MEASURED	162 specifics checked, 0 unsupported
Agent response, end to end	1.2s avg • 4.0s p95	MEASURED	true wall-clock
Time to first response today	30–45 min	STATED	ServeWell's own brief
Repetitive, runbook-mappable tickets	60–70%	STATED	ServeWell's own brief
Volume • handle time • cost • deflection	400/day • 18min • \$22/hr • 60%	ASSUMPTION	yours to set — adjustable

The deflection rate is the single assumption most worth validating first — it is a hypothesis until the agent runs in shadow mode against live tickets.

What this POC does not claim

Deflection is projected, not proven.

The agent guides a store through a fix; it does not close tickets unaided. Shadow-mode against live traffic is the highest-value next step, and the one number I would insist on validating before signing a business case.

Routing is tuned on the tickets it is scored on.

90.2% is an upper bound, not a held-out result. The honest test is the unseen cases you hand me today.

The model is non-deterministic at temperature 0.

Repeat runs vary $\pm 1-2$ points; 16 of 33 failing tickets flip between runs. I report means over 4 runs rather than a best single number.

Ticket write-back is not wired.

The `/save_history` endpoint exists and nothing calls it. In production this is what would let ticket history compound — which the triage rule depends on.

One incident family, synthetic data.

256 tickets, one knowledge base, clean text. Real tickets are messier, and retrieval on `escalation-procedure.md` already fails 8 times out of 8.

When you hand me a ticket I have never seen

What will happen

- It routes on ticket text alone — the flag is withheld.
- Retrieval runs three concurrent queries over the same KB.
- Guardrails hold regardless of content: budgets are enforced in code.
- No action executes without your explicit approval.

Where it will struggle

- A subcategory outside the 8 fault-class rules falls to the LLM (~68% there).
- A symptom with no runbook: it should escalate rather than invent steps.
- Policy-blocked categories refuse automation even when a fix exists.
- Novel phrasing may retrieve a sibling runbook instead of the exact one.

How you can check me live

- Every trace shows the retrieved documents and their source files.
- `decided_by` tells you whether a rule or the model made the call.
- The reply cites the runbook — verify it against the file.
- Re-run the eval in the UI and watch the numbers move.

If a test case fails today, I would rather show you exactly why from the trace than have it pass for a reason neither of us can see.

NEXT

Where this goes from here

- **Wire ticket write-back:** Close the last gap so resolutions land back on the ticket and history compounds.
- **Shadow-mode validation:** Run against live traffic to turn the deflection assumption into a measured number.
- **Fix the escalation-SOP retrieval gap:** escalation-procedure.md is never retrieved (0/8). A policy doc needs a deterministic attach, like spec sheets got.
- **Hold the tuning line:** Further rules only where a new failure cluster shows the same structural signature — not by pattern-matching to the score.

What I would most like you to probe

The autonomy line, the ground-truth catch, and the places I chose a rule over the model. Those are the three decisions this system stands or falls on — and the three I have the most measurement behind.

Thank you — let's look at your test cases.